

You are absolutely right, and it's a very common roadblock when upgrading to the Minima. MCUFRIEND_kbv heavily relies on direct port manipulation tailored for older AVR chips. The UNO R4 uses a completely different Renesas RA4M1 processor architecture, meaning those low-level registers do not exist.

To fix this, we need to swap the graphics engine to the **GFX Library for Arduino** (maintained by *moononournation*). This library includes an Arduino_UNOPAR8 class specifically written to map an 8-bit parallel bus onto the UNO R4's Renesas pins.

1. New Library Requirement

Remove MCUFRIEND_kbv and install this via the Arduino Library Manager:

- **GFX Library for Arduino** (Search exactly for this; it handles both the bus and the ILI9486 driver).

2. The Updated R4 Minima Code

The logic for your RTC and W25Q flash memory remains exactly the same. The only changes are in how we initialize the screen and handle the display object (using pointer syntax tft-> instead of tft.).

```
#include <SPI.h>
#include <Arduino_GFX_Library.h> // Replaces MCUFRIEND_kbv

// --- DISPLAY SETUP ---
// Arduino_UNOPAR8 handles the 8-bit parallel bus specifically for UNO
// R4 / Renesas boards
Arduino_DataBus *bus = new Arduino_UNOPAR8();
// Initialize the ILI9486 driver. (A4 is the standard Reset pin for
// Uno display shields)
Arduino_GFX *tft = new Arduino_ILI9486(bus, A4, 0 /* rotation */,
false /* IPS */);

// Color definitions (Standard 16-bit RGB565)
#define BLACK    0x0000
#define WHITE    0xFFFF
#define GREEN    0x07E0
#define CYAN     0x07FF

// --- W25Q FLASH MEMORY SETUP ---
#include <Adafruit_SPIFlash.h>
#define FLASH_CS_PIN 10
Adafruit_FlashTransport_SPI flashTransport(FLASH_CS_PIN, &SPI);
Adafruit_SPIFlash flash(&flashTransport);

// --- RTC SETUP ---
#include "RTC.h"
RTCtime currentTime;
int lastSecond = -1;
```

```

void setup() {
    Serial.begin(115200);

    // 1. Initialize Display
    // Using pointer syntax (->) because tft is now a pointer object
    if (!tft->begin()) {
        Serial.println("Display init failed!");
    }
    tft->setRotation(1); // Landscape mode
    tft->fillScreen(BLACK);

    // 2. Initialize RTC
    RTC.begin();
    // Set startup time: Sept 2nd, 2026, 14:30:00
    RTCTime startTime(2, Month::SEPTEMBER, 2026, 14, 30, 00,
    DayOfWeek::WEDNESDAY, SaveLight::SAVING_TIME_INACTIVE);
    RTC.setTime(startTime);

    // 3. Initialize Flash Memory
    tft->setCursor(10, 10);
    tft->setTextColor(WHITE);
    tft->setTextSize(2);

    if (!flash.begin()) {
        tft->println("Flash Init Failed!");
        return; // Halt if memory isn't found
    }

    // 4. Write data to W25Q Memory
    uint32_t address = 0;
    char writeData[] = "Data from W25Q on R4!";

    flash.eraseSector(address / 4096);
    flash.writeBuffer(address, (uint8_t*)writeData, strlen(writeData) +
1);

    // 5. Read data back from W25Q Memory
    char readBuffer[64] = {0};
    flash.readBuffer(address, (uint8_t*)readBuffer, sizeof(readBuffer));

    // 6. Display the read data in the center of the screen
    tft->setCursor(20, tft->height() / 2);
    tft->setTextColor(GREEN);
    tft->setTextSize(2);
    tft->print(readBuffer);
}

void loop() {

```

```

RTC.getTime(currentTime);

if (currentTime.getSeconds() != lastSecond) {
    lastSecond = currentTime.getSeconds();

    char timeStr[10];
    sprintf(timeStr, "%02d:%02d:%02d", currentTime.getHour(),
currentTime.getMinutes(), currentTime.getSeconds());

    // Calculate width to right-align the clock
    int textWidth = 8 * 2 * 6; // 8 chars * text size 2 * 6 pixels
width per char
    tft->setCursor(tft->width() - textWidth - 10, 10);

    // Draw text with a black background to prevent screen flicker
    tft->setTextColor(CYAN, BLACK);
    tft->setTextSize(2);
    tft->print(timeStr);
}
}

```

Since the standard 8-bit parallel pins for the ILI9486 shield are D2-D9 and A0-A4, this perfectly leaves the R4 Minima's hardware SPI pins (D10-D13) completely free to handle your Winbond W25Q communication without any bus collisions.

[Setting up an ILI9486 Display with the UNO R4 Minima using Arduino_GFX](#) This video demonstrates the specific Arduino_GFX driver setup running an ILI9486 display natively on the R4 Minima's Renesas processor. *YouTube video views will be stored in your YouTube History, and your data will be stored and used by YouTube according to its [Terms of Service](#)*